Step 3 — Multiple MCP Servers at Once, With Tool Namespacing — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Almost no new code — the pool already loops over `self.servers`, namespaces, and routes. What's new is a **second server** in the config and a task that needs both.

- Write a second example server, `servers/files_server.py` (`list_files`, `read_file`), the same `MCPServer` shape as the notes server.
- Add it to `SERVERS` in `servers_config.py`.
- Confirm the pool gives every tool a name like `notes__search_notes` / `filesystem__list_files`, and keeps `routing[namespaced] = (server, tool)`.

The gotcha to internalise: OpenAI's function-name field allows only letters, digits, `_`, `-`. A `.` separator (`notes.search_notes`) is rejected by the API. Use `__`. That's why the model sees the namespaced name and your routing dict turns it back into `(server, tool)` before dispatch.

Hint 2 — The plan, and almost the whole thing

```
servers/files_server.py:
    MCPServer("files-server"); @mcp.tool() list_files, read_file

servers_config.py:
    SERVERS = [ {notes...}, {filesystem...} ]   # two entries now

(no pool change — connect_all already does this per server:)
    namespaced = name + "__" + tool.name
    routing[namespaced] = (name, tool.name)

main.py:
    TASK = "...notes about the deploy process, and also list the files..."
```

```
    run it; print each step's namespaced tool name

test_agent.py (+ case 3):
    connect([NOTES, FILES]); run the both-servers task
    assert "notes__search_notes" in tools_used
    assert "filesystem__list_files" in tools_used
```

Check the **tools used**, not the final answer text — a model can produce a plausible answer while calling the wrong server, and only the routing check catches that.

The whole file set, runnable, is in the Solution.